P2P Payment Ledger

Final Project Requirements

Simplified UPI / Wallet with Delayed P2P Settlement and Check Please

This document consolidates the original P2P Payment Ledger specification with the agreed Check Please feature. It is the requirements baseline for implementation.

Configuration	Requirement
Java	Java 25
Backend framework	Spring Boot 4.1.2
Backend architecture	Spring Data JPA, Spring Security with JWT, Spring Scheduler
Database	PostgreSQL
Frontend	React
Build and configuration	Maven pom.xml and application.properties
Explicit exclusions	No YAML configuration and no Flyway

This document intentionally excludes a project timeline and a prescribed REST API endpoint list. API design is left to the implementation agent, subject to the functional requirements below.

Contents

- 1. Project overview and scope
- 2. User registration and authentication
- 3. Wallet management
- 4. P2P transfers and delayed settlement
- 5. Fraud controls
- 6. Transaction history and statistics
- 7. Notifications
- 8. Scheduled background jobs
- 9. Admin panel and audit
- 10. Check Please — merchant-specific payment-intent reservations
- 11. Database requirements
- 12. Security, consistency and operational requirements
- 13. Acceptance criteria

1. Project overview and scope

Build a peer-to-peer digital wallet where users can register, load simulated money, transfer money to other users, and track transactions. P2P transfers use an ACH-style delayed settlement model. The system detects basic fraud patterns, notifies users, runs background settlement and housekeeping jobs, and provides an admin monitoring view.

The application also includes Check Please: a feature inspired by the user's requested CRED.ai-style experience. A user can declare an intended payment to a registered merchant, set a maximum amount and choose an expiry. The system recognizes an eligible payment associated with that reservation and applies the agreed payment-risk handling, while retaining critical security controls.

Technology stack

Area	Requirement
Language	Java 25
Backend	Spring Boot 4.1.2; Spring Data JPA; Spring Security with JWT; Spring Scheduler
Database	PostgreSQL
Frontend	React
Build/configuration	Maven; application.properties
Configuration constraints	Do not use YAML configuration or Flyway

Scope boundaries

- Adding money to a wallet is simulated; there is no real bank integration.
- Merchant payments and merchant settlement are simulated in the initial version; there is no payment-gateway integration.
- P2P transfers use delayed settlement. Merchant payments use a separate lifecycle and are captured immediately in the MVP after authorization succeeds.
- The implementation agent may choose suitable API routes, request/response DTOs, and internal design, provided all requirements and access-control rules are met.

2. User registration and authentication

- A user registers with name, email, phone number, password, and a four-digit transaction PIN.
- Successful registration automatically creates exactly one ACTIVE wallet with an initial balance of ■0.
- Users log in with email and password. The system issues a JWT after successful authentication.
- Support USER and ADMIN roles. Users may access only their own data; admins may access authorized administration functions.
- The transaction PIN is separate from the login password and is required for money transfers and applicable merchant payments.
- Track consecutive failed login attempts. Lock an account after five consecutive failures for 30 minutes.
- Automatically unlock eligible accounts and notify the affected user.
- Store passwords and transaction PINs using secure password hashing. Never store plaintext credentials or log secrets.

3. Wallet management

- Every user has exactly one wallet, created during registration.
- Users can load simulated money; validate the amount and record the operation appropriately.
- Display total wallet balance, available balance, reserved balance, and today's total spending.
- Wallet statuses are ACTIVE, INACTIVE, and FROZEN.
- ACTIVE wallets can send and receive eligible funds.
- INACTIVE wallets block outgoing transactions. Incoming P2P settlements are declined, the transfer is marked DECLINED, the sender is refunded, and the sender is notified. The owner may reactivate an INACTIVE wallet.

- FROZEN wallets block outgoing transactions. Incoming P2P settlements are declined, the sender is refunded and notified. Only an admin may freeze or unfreeze a wallet.
- Sending from an INACTIVE or FROZEN wallet must fail with a clear status-related error.
- Available balance is the amount currently spendable. Reserved funds must not be spendable through unrelated transactions.
- Use Java BigDecimal and PostgreSQL NUMERIC for money; do not use floating-point types.

4. P2P transfers and delayed settlement

- Users can search for registered recipients by phone number or email.
- To send money, the user selects a recipient, enters a positive amount, optionally enters a remark, and confirms using their four-digit transaction PIN.
- Each request carries a unique client-generated request_id. Repeating the same request_id returns the original result and must not process the payment again.
- On initiation, the sender's available balance is reduced immediately, a sender DEBIT ledger entry is created, and the transfer status becomes PENDING.
- The receiver is not credited or notified about the pending transfer before settlement.
- Record a settlement_due_date three business days after initiation. Exclude weekends; the holiday-calendar policy must be documented.
- A scheduled settlement job credits the receiver when the transfer becomes eligible, creates the receiver CREDIT ledger entry, changes the status to SUCCESS, and notifies both parties.
- If settlement fails, mark the transfer FAILED, refund the sender exactly once, notify the sender, and continue processing other eligible transfers.
- If the receiver wallet is INACTIVE or FROZEN at settlement time, mark the transfer DECLINED, refund the sender exactly once, and notify the sender.
- Keep transaction status transitions valid and auditable.

5. Fraud controls

- Daily P2P transaction limit: ■50,000 per user. A request exceeding the limit is blocked and a fraud alert is created.
- Duplicate detection: block an exact same-amount transfer to the same recipient within two minutes of a previous transaction; create a fraud alert.
- High-value threshold: a P2P transfer exceeding ■8,000 is allowed to enter PENDING settlement but creates a fraud alert for admin review.
- Before settlement, an admin may block a flagged pending transfer. The transfer becomes FAILED, the sender is refunded, and both parties are notified.
- If no block action occurs by settlement time, the transfer settles normally, subject to mandatory checks.
- All fraud alerts must be visible in an admin queue.
- Check Please must not disable critical security controls. Risk policies must distinguish P2P transfers from merchant payments instead of blindly applying or bypassing every P2P rule.

6. Transaction history and statistics

- Show a mini-statement containing the last 10 transactions.
- Provide paginated full history with date range, type (sent/received/all), and status (success/pending/failed/all) filters.
- Pending P2P transfers show their status, expected settlement date, and days remaining.
- Transaction details include related ledger entries. A pending P2P transfer has only the sender DEBIT entry until settlement.
- Show personal statistics: total sent, total received, and this month's transaction count.
- Users must only access their own transaction and ledger data.

7. Notifications

- Generate in-app notifications for transfer initiation, successful settlement, transfer failure or decline, blocked transactions, fraud alerts, account lock, and account unlock.
- Notify the sender and receiver after a P2P transfer settles successfully.
- Notify the sender when a transfer fails or is declined and refunded.
- Display an unread count on a notification bell. Users can view notifications and mark them as read.
- Check Please notifications cover reservation creation, merchant payment success or failure, cancellation, and expiry.

8. Scheduled background jobs

- ACH settlement processor runs hourly and processes PENDING transfers whose settlement due date has arrived.
- For each successful settlement, credit the receiver, write the CREDIT ledger entry, update status and notify both parties.
- For an individual settlement failure, mark FAILED, refund the sender exactly once, log the failure, and continue with other transactions.
- Daily limit reset runs at midnight. Document the timezone policy.
- Account auto-unlock runs every five minutes and unlocks accounts whose 30-minute lock period has elapsed, notifying each affected user.
- Daily summary and reconciliation run at 11 PM. Summarize initiated transfers and amounts, settled transfers and amounts, fraud alerts, new registrations, and total pending settlement amount.
- Reconciliation compares debit and credit ledger totals for settled P2P transfers. Pending transfers are excluded. Any mismatch creates a critical admin notification.
- Check Please expiry processing releases remaining reserved funds and marks expired reservations without releasing funds twice.
- Every job run logs start time, end time, success/failure, records processed, and error details when applicable.
- Each settlement batch records settled count, failed count, and total amount settled.

9. Admin panel and audit

- Dashboard shows registered users, today's initiated and settled transfer counts, today's transaction volume, total pending settlement amount, pending fraud alerts, and a 30-day daily transaction-volume chart.
- User list shows wallet balance, wallet status, registration date, and recent transactions. Admins can freeze or unfreeze wallets.
- Transaction monitoring shows all transactions with date, status, and amount filters, including ledger details and clear pending-state presentation.
- Settlement monitor shows pending transfers, due-today, due-tomorrow and overdue counts. Authorized admins may manually trigger settlement for a specific eligible transaction.
- Fraud review shows pending alerts. Admins can block or dismiss eligible high-value alerts; alerts for settled transactions are read-only.
- Job monitor shows the last 20 scheduled job runs and their results.
- Reconciliation shows settled debit versus credit totals, pending settlement amount and mismatches.
- Admins can register merchants and activate, deactivate or suspend them.
- All admin actions must be written to an audit log.

10. Check Please — merchant-specific payment-intent reservations

Check Please is inspired by the requested CRED.ai-style idea of declaring an intended merchant payment in advance so the system can recognize it and reduce the risk that an otherwise eligible payment is interrupted by ordinary risk heuristics. In this project, the feature is defined as a merchant-specific reservation plus explicit payment-intent metadata. It is not a guarantee that every payment will succeed and must not override critical security blocks.

10.1 Confirmed requirements

Aspect	Requirement
Merchant scope	Only registered merchants with ACTIVE status can be selected for new reservations and eligible payments.
Maximum amount	The user reserves up to a maximum amount. A qualifying payment may be less than or equal to the remaining reserved amount.
Expiry	The user selects an expiry timestamp. The system validates that it is in the future and within configurable duration limits.
Reserved funds	The reserved amount is set aside from spendable balance and cannot be used for unrelated P2P or merchant transactions.
Fraud behaviour	A valid reservation is a positive signal of declared payment intent and receives special handling under the configured merchant-risk policy. Mandatory security checks remain active.
Merchant payment lifecycle	Merchant payments are separate from delayed P2P transfers and are captured immediately in the simulated MVP after successful authorization.
Failure	If payment is rejected before capture, reserved funds are not debited. The reservation remains available for retry if still active and valid.
Successful payment	A successful payment consumes the reservation, debits the payment amount, releases any unused reserved amount, and marks the reservation CONSUMED.
Reuse	A reservation can authorize only one successful merchant payment. The unused remainder is released after capture.

10.2 User flow

- The user selects an active registered merchant.
- The user enters a maximum amount and chooses an expiry timestamp.
- The system validates ownership, merchant status, expiry, amount and sufficient available balance.
- The system reserves the funds, reducing the available balance by the reserved amount.
- When a merchant payment request is initiated, the system verifies the merchant identity, reservation ownership, status, expiry, amount, wallet status, transaction PIN and mandatory security checks.
- If eligible and approved, the system atomically captures the payment, records the ledger entry, consumes the reservation and releases its unused remainder.
- If payment is blocked or fails before capture, the reserved amount is not debited. The active reservation can be retried if still valid.
- When the user cancels a reservation or its expiry is reached, remaining reserved funds are released exactly once.

10.3 Reservation states

- ACTIVE → CONSUMED after one successful eligible payment.
- ACTIVE → CANCELLED when cancelled by its owner.
- ACTIVE → EXPIRED when the expiry timestamp is reached.
- CONSUMED, CANCELLED and EXPIRED reservations cannot authorize further payments.

10.4 Validation and safety rules

- A user cannot reserve more than the wallet's available balance.
- The reservation is bound to one user, one wallet and one merchant.
- The payment merchant must match the reservation merchant.
- The payment amount must be positive and no greater than the reservation's remaining amount.
- An inactive or suspended merchant cannot receive new reservations or eligible payments.
- An inactive or frozen wallet cannot make a merchant payment.
- Critical security blocks override Check Please, including invalid authentication, invalid PIN, invalid ownership, invalid merchant identity, compromised-account indicators or other mandatory security policy blocks.

- Use idempotency to prevent repeated requests from debiting twice.
- Use database transactions and appropriate row locking to prevent concurrent payments from consuming the same reservation or funds.
- Failed payment attempts do not consume the reservation if no capture occurred. If the reservation expires or is cancelled, it cannot be reused.
- Ordinary P2P fraud rules must not be blindly reused for merchant payments. Define explicit merchant-payment risk rules and document which non-critical checks can be adjusted for valid reservations.

10.5 User experience

- Provide a list/search of active registered merchants.
- Allow users to create reservations with a maximum amount and chosen expiry.
- Show merchant, maximum amount, remaining reserved amount, expiry, and reservation status.
- Allow users to cancel active reservations.
- Show merchant payment outcomes and reservation-related status in transaction history.

10.6 Accounting expectations

- The wallet must distinguish total balance, reserved balance and available balance.
- Available balance = total wallet balance – reserved balance.
- Creating a reservation increases reserved balance but does not create new money or change total wallet balance.
- Successful payment decreases total wallet balance by the captured amount and releases the full remaining reservation from reserved balance.
- Cancellation and expiry do not change total wallet balance; they reduce reserved balance and increase available balance.
- All wallet, reservation, payment and ledger changes must commit atomically. Failed operations must not leave partial financial changes.

11. Database requirements

The original specification lists 11 tables. Retain those tables and add storage for registered merchants, Check Please reservations and merchant payment records. The implementation agent determines the exact schema and relationships based on the repository and domain model; the following is a logical inventory, not a mandatory SQL/API design.

Table / logical entity	Purpose
users	User profile, credentials, role and account-lock data
login_attempts	Failed login tracking
wallets	Wallet total balance, reserved balance and status
transactions	P2P transfers, status and settlement date
ledger_entries	Financial debit and credit records
fraud_alerts	Fraud rules and admin review
notifications	In-app messages and read status
audit_log	Administrative actions
scheduled_job_log	Background job execution logs
daily_summary	Daily financial and activity aggregates
settlement_batches	Per-run settlement counts and amounts
merchants (additional)	Registered merchant identity and status
check_please_reservations (additional)	Merchant-specific maximum amount, remaining amount, expiry and state
merchant payments (additional logical entity)	Merchant payment requests, reservation association and outcome

- Use foreign keys, uniqueness constraints, CHECK constraints and indexes where appropriate.

- Use UUID or the existing project's consistent ID convention.
- Use NUMERIC for monetary values and BigDecimal in Java.
- Use transaction boundaries and locking for concurrent financial operations.
- The final schema must preserve the original P2P ledger and settlement behaviour while adding Check Please.

12. Security, consistency and operational requirements

- Use Spring Security with JWT and role-based authorization.
- Never trust client-supplied user IDs for ownership checks.
- Hash passwords and transaction PINs; never log secrets.
- Validate input and return consistent error responses.
- Use @Transactional for financial workflows and appropriate row locking or equivalent concurrency controls.
- Enforce idempotency using database constraints and return original results for retries.
- Make settlement, refunds, reservation expiry, cancellation and capture safe to retry.
- Prevent duplicate debits, duplicate credits, duplicate refunds and duplicate reservation releases.
- Keep P2P delayed settlement separate from merchant payment capture.
- Log job execution and administrative actions.
- Document timezone, business-day and reservation-expiry policies.
- This is a simulated educational wallet and must not be represented as a production-ready regulated payment service.

13. Acceptance criteria

- Registration creates exactly one ACTIVE wallet with ■0 initial balance.
- JWT authentication and USER/ADMIN access controls work.
- Five consecutive failed login attempts cause a 30-minute lock; auto-unlock and notification work.
- P2P transfers debit immediately and settle after three business days.
- Repeated P2P request IDs do not cause duplicate transfers or debits.
- Daily limits, duplicate detection and high-value fraud alerts work as specified.
- Settlement failures and receiver wallet rejections refund the sender exactly once.
- Users can view paginated transaction history, ledger details, notifications and personal statistics.
- Admin dashboard, fraud review, job monitoring, settlement monitoring, audit and reconciliation are available.
- Users can reserve funds for an active registered merchant with a maximum amount and user-selected expiry.
- Reserved funds cannot be spent on unrelated transactions.
- A valid eligible merchant payment consumes the reservation once and releases unused reserved funds.
- Expired or cancelled reservations release remaining reserved funds exactly once.
- Check Please does not bypass critical security controls.
- Merchant payment retries are idempotent and concurrent operations preserve wallet and ledger consistency.
- All scheduled jobs and admin actions are logged.
- Automated tests cover ownership, expiry, concurrency, refunds, payment failure, fraud policy and reconciliation.

Implementation note: This document intentionally contains no project timeline and does not prescribe REST API endpoints. The implementation agent should inspect the existing workspace and decide API routes, DTOs, detailed schema, and task sequencing while preserving the functional requirements and business rules above.